VigiClaw: Action-Triggered State Verification for Robust Long-Horizon Robotic Manipulation

Pengxiang Lu, Ning Kang, Chongjin Gao, Yi Wang* and Zhenyu Lu*

Abstract— **Recent advances in Large Language Models (LLMs) and Vision-Language-Action (VLA) models have improved language-conditioned robotic task planning, yet long-horizon manipulation remains fragile when physical execution is disturbed. Existing OpenClaw-style systems provide modular skill scheduling and execution feedback, but they do not explicitly verify whether each completed atomic action has achieved its intended effect. In this work, we propose VigiClaw (Vigilant OpenClaw), an OpenClaw-centered manipulation framework with a vision-based Action-Triggered State Verification mechanism. After each atomic action, VigiClaw reconstructs the visual world state and triggers rule-guided replanning upon detecting anomalies. We evaluated VigiClaw on eight ManiSkill tabletop manipulation tasks, including perturbed long-horizon block placement and stacking tasks. Experimental results show that, facing disturbances, VigiClaw achieves success rates of 84% and 72% in placement and stacking, respectively, while baselines lacking visual detection fail completely, indicating that Action-Triggered State Verification improves recovery under disturbances.**

I. Introduction

In recent years, the rapid development of Large Language Models (LLMs) and Vision-Language-Action (VLA) models has enabled robots to comprehend natural language instructions and perform high-level task planning with unprecedented flexibility. Code-as-Policies [1] invokes perception/control APIs to generate code programs, while SayCan [2] generates the next action to execute; meanwhile, VLA models [3], [4], [5] are capable of mapping natural language instructions and visual observations directly to robot actions. The common advantage of these methods lies in their ability to allow robots to perform task reasoning within an open language space, rather than relying on manually designed finite-state automata.

Despite these advances, long-horizon physical execution remains extremely fragile. The real world is dynamic—objects may shift position, grasps may slip, and low-level action primitives often fail to achieve their intended "post-conditions". To address this, recent research has increasingly focused on execution monitoring and failure recovery mechanisms. For example, Code-as-Monitor [6] uses a VLM to generate spatiotemporal constraints and executable monitoring code based on task instructions; at runtime, this code checks whether geometric elements in visual observations satisfy these constraints to detect anomalies or failures. Guardian [7] inputs the robot's current multi-view visual observations, task instructions, and the description of the executing subtask into a fine-tuned VLM, directly using it in a visual question-answering manner to determine if there is a planning or execution error, outputting specific failure categories along with explanatory reasoning. Although these two methods enhance the system's anomaly awareness, they stop at anomaly detection and diagnosis, merely outputting textual alerts or failure labels, making it difficult to form a complete closed-loop control from detection to recovery.

In contrast, recent robotic frameworks centered around OpenClaw [8] offer potential solutions to this issue. For example, ROSClaw [9] integrates OpenClaw with ROS 2, achieving control of robots by foundation models through capability discovery and pre-execution verification; OpenGo [10] builds on this to create an embodied robotic dog system, using execution feedback for real-time skill scheduling and closed-loop adaptation. Although these two works have made progress in the instruction-to-action mapping loop, they both lack explicit post-action execution verification mechanisms, making it difficult to definitively confirm whether the intended state

This work was supported by the Fundamental Research Funds for the Central Universities 2025ZYGXZR057 and the Guangdong Provincial Key Area S&T Program 2025B0909030001.

Pengxiang Lu, Ning Kang, Yi Wang and Zhenyu Lu are with the School of Automation Science and Engineering, South China University of Technology, Guangzhou, China. Chongjin Gao is with the Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University, Xi'an, China.

*Corresponding authors: Yi Wang and Zhenyu Lu (e-mail: luzhenyu@scut.edu.cn).

changes have truly occurred upon action completion.

To address this, we propose VigiClaw (Vigilant OpenClaw), an OpenClaw-centric manipulation system that introduces an Action-Triggered State Verification mechanism. After the completion of each atomic action (e.g., grasping, placing), VigiClaw visually inspects whether the expected state change has occurred, triggering rule-guided replanning upon detecting anomalies. Unlike methods requiring continuous VLM monitoring, VigiClaw grounds its verification logic at the atomic action level, enabling robust responses to external disturbances without the need for continuous inference. The main contributions of this paper are as follows:

- We propose a vision-based Action-Triggered State Verification mechanism that enables robust detection of external disturbances.
- We present VigiClaw, a robotic manipulation framework centered on OpenClaw, that integrates high-level task reasoning, visual world-state estimation, parameterized skill execution, and rule-guided recovery within a unified execution loop.
- Facing external disturbances in long-horizon tasks, VigiClaw achieves success rates of 84% and 72% on block placement and stacking, in contrast to the complete failure of baselines without visual detection.

II. Methodology

A. Overview of the System

VigiClaw adopts a server-client architecture in which the server runs the ManiSkill [11] simulator. In VigiClaw, inputs are natural language commands and RGB-D observations, and the outputs are sequences of parameterized skills that encompass both task execution and recovery actions, continuing until task success or the retry limit (three times) is reached. As shown in Fig. 1, the client comprises the perception and decision-making layers, while the server handles the execution layer, together forming a complete closed-loop system. The specific execution workflow is as follows: First, OpenClaw performs natural language inference on the task instruction via API calls to determine which skill (grasping, placing, or pushing) should be used to manipulate the object. Subsequently, RGB and depth images are obtained from the simulation environment. YOLO [12] performs instance segmentation on the RGB image to obtain a binary mask, which combined with the depth map, is used to reconstruct a 3D point clouds for calculating the pose of all objects in the world coordinate system. These object poses are then fed into OpenClaw, which passes them along with the weight files as parameters to the corresponding DMP [13], [14] trajectory generation script. The generated trajectory consists of a sequence of points $[x, y, z, \alpha, \beta, \gamma, g]$, which is transmitted to the server and executed by the controller in the simulation environment to drive the robotic arm.

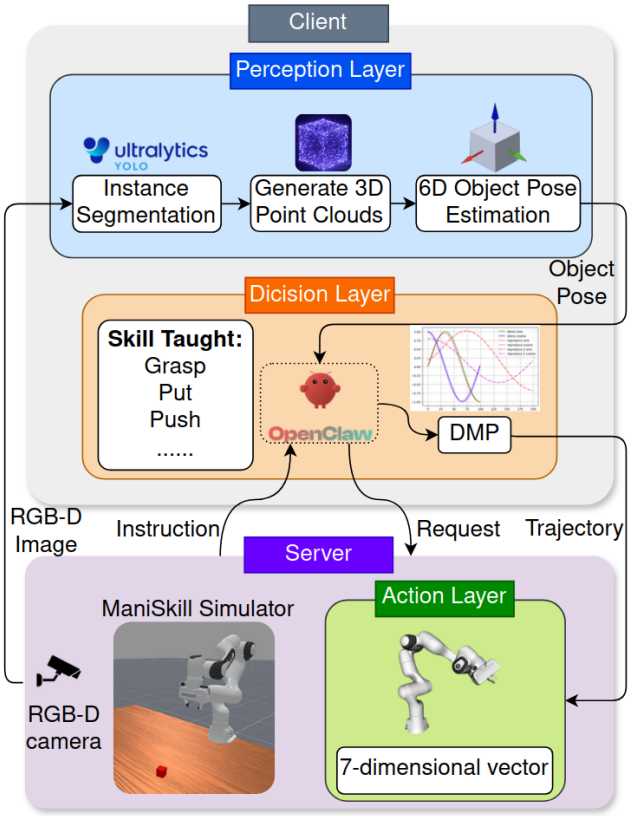


Fig. 1. Block diagram of the VigiClaw system architecture. The perception layer extracts object poses from RGB-D images, OpenClaw generates trajectories via DMP weights, and the execution layer controls the robot in simulation.

Fig. 2 illustrates the decision-making workflow of VigiClaw. First, the system receives the task instruction, acquires the initial RGB-D image, and begins task execution. OpenClaw decomposes the high-level task into atomic steps such as grasping, lifting, placing, or pushing. Intermediate checks and end-of-task verification are performed, triggering state verification (detailed later). If an anomaly is detected, OpenClaw uses a retry strategy. This cycle continues until no anomaly is detected or the retry count exceeds the limit. Upon completion, functional scripts are called to terminate the task. The server then evaluates task success, updates the success rate statistics, and reinitializes the simulator scene.

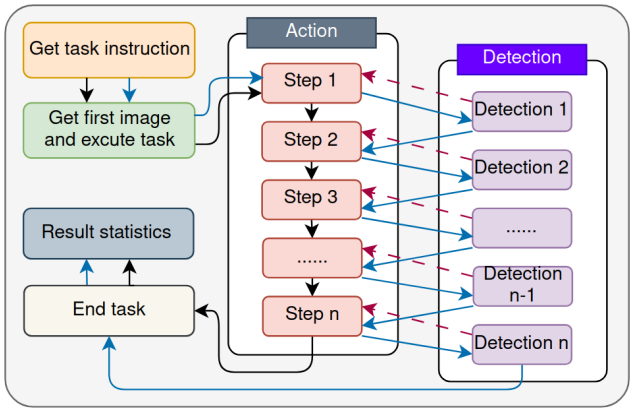


Fig. 2. OpenClaw decision-making workflow. It includes the entire process of the task from start to finish, where black arrows represent no state verification, blue arrows represent state verification, and red arrows represent retries.

B. Skill Library and DMP-Based Skill

Unlike Code-as-Policies [1] and VoxPoser [15], which allow LLMs to generate executable code snippets at runtime, VigiClaw relies on a fixed set of predefined functional APIs. The entire process is implemented by invoking predefined functional APIs. The functionalities of these APIs are as follows:

- Retrieving server status and terminating tasks.
- Retrieving task instructions.
- Acquiring object poses.
- Generating DMP trajectories.

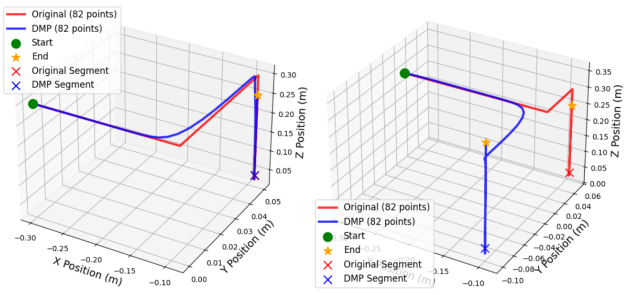


Fig. 3. Comparison chart of original trajectory and DMP-generated trajectory. The red line represents the teaching trajectory, and the blue line represents the trajectory generated by DMP. The left panel illustrates the trajectory reproduction without altering intermediate and goal points, while the right panel demonstrates trajectory generalization achieved by modifying these key points.

In the skill documentation (SKILL.md) for OpenClaw, we explicitly define the following basic rules to ensure the standardization and reliability of system operation:

- Restricting code execution to predefined scripts.
- Defining basic skill invocation procedures.
- Enforcing a standard workflow of instruction fetching, task execution, and termination.
- Defining the world coordinate frame orientation.
- Specifying rule-guided recovery strategies.

A demonstrator manually operates the robotic arm in the simulation environment using a joystick to collect demonstration trajectories, which are then learned via DMP to generate the corresponding weight files. Currently, weight files are available for three primitive actions: grasping, placing, and pushing. The DMP script generates generalized trajectories from weight files and key points, preserving the original demonstration's characteristics. For multi-key-point actions such as grasping and lifting, we adopt a segmented DMP method: the trajectory is partitioned at key points, fitted separately, and then merged. Considering the multi-dimensional nature of spatial trajectories, each dimension is processed independently before concatenation. Since DMPs have difficulty modeling the discrete gripper state, the system directly uses the gripper state values from the original trajectory points. As shown in Fig. 3, the generated trajectories not only faithfully reproduce the demonstration well but also retain its motion characteristics after key-point modification.

C. Principle of Object Pose Calculation

In the ManiSkill simulation environment, we configured a fixed RGB-D camera overlooking the desktop at a 45-degree downward angle, synchronously capturing RGB and depth images at a resolution of 512×512. We meticulously collected and annotated 260 images in custom scenes to construct a dedicated dataset; during training, the input images were uniformly resized to 512×512 and the model was trained for 300 epochs with a batch size of 16. Fig. 4 illustrates the pipeline to estimate the pose of the object.

For the acquired RGB-D image, via YOLO instance segmentation, we produce the binary mask mask $M(u,v)$ of the target object in the image:

$$M(u,v) = \begin{cases} 1, & (u,v) \in \text{target object} \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

By combining the segmentation mask with the depth map and using camera intrinsics, we back-project the target pixels into 3D space to form the object's point clouds. For each pixel (u,v) where the mask $M(u,v) = 1$, the corresponding 3D point $\mathbf{p}_c = [X_c, Y_c, Z_c]^T$ in the camera coordinate system is calculated as follows:

$$X_c = \frac{(u - c_x) \cdot Z_c}{f_x}, \quad Y_c = \frac{(v - c_y) \cdot Z_c}{f_y} \tag{2}$$

where (f_x, f_y) denotes the focal length, (c_x, c_y) represents the principal point, and Z_c is obtained from the depth image.

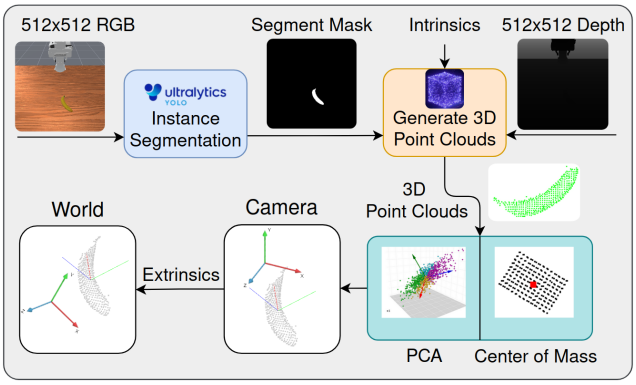


Fig. 4. Process for Obtaining 6D Object Pose. YOLO instance segmentation yields a binary mask, which is combined with the depth map and camera intrinsics to back-project the object's point cloud. The object's position is determined by the point cloud centroid, and its orientation is computed via PCA to obtain the 6D pose in the camera coordinate system.

Finally, these points make up the point clouds $P_{\text{cam}} = \{\mathbf{p}_c^{(i)}\}_{i=1}^{N}$ in the camera coordinate system. The object position is determined by calculating the centroid of the point clouds, and the object pose is analyzed using PCA [16].

For tabletop planar manipulation tasks, the robot end-effector position is specified by the Cartesian coordinates (x, y, z), while the orientation is governed solely by the yaw angle α.

D. Action-Triggered State Verification and Rule-Based Recovery

An anomaly is defined as a mismatch between the intended outcome of a robotic action and the actual observed state, or an unintended displacement of non-target but task-relevant objects. Specifically, anomalies include failing to grasp an object despite attempting to do so, or failing to place an object within a designated target region. Here, "Action-Triggered" signifies that state verification is activated upon the completion of each atomic action (excluding the initial frame) to acquire and assess the scene state.

The visual detection mechanism operates as follows: Upon completion of each action, the world coordinates of all objects are calculated and recorded to a file. These coordinates are subsequently compared against the recorded state of the previous step. The specific verification logic in the OpenClaw skill file (SKILL.md) is as follows:

- **Grasping:** If the distance between the object's coordinates and the end-effector position after lifting exceeds a predefined threshold, the grasp is deemed a failure.
- **Placing:** If the distance between the detected object pose and the expected target pose exceeds a threshold, placement is considered unsuccessful.
- **External Disturbance:** If the positional displacement of non-target objects between consecutive steps exceeds a threshold, an external interference is detected.

The threshold mentioned above is set to 0.05 m. This value is empirically determined based on the average dimensions of the object and the positional accuracy of the robotic arm within the ManiSkill simulation environment. We realize rule-guided recovery strategies within the OpenClaw skill library by explicitly enumerating recovery rules for different anomalies, enabling OpenClaw to autonomously select appropriate corrective actions. Specifically, for grasping anomalies, OpenClaw adjusts the grasp yaw angle and retries the grasp; for placing anomalies, it re-executes the grasp-and-place procedure; for anomalies involving non-target objects, the system updates the object's pose to reflect its current location, thereby avoiding reliance on outdated state information. To prevent infinite loops, the system enforces a retry limit of three attempts for each atomic action before terminating the task or escalating the failure. Furthermore, if the target object is not detected after an anomaly, the system treats this as an unrecoverable failure and ends the task directly.

III. Experiment and Result

A. Experimental Design

This experiment is conducted on the OpenClaw platform (v2026.3.13) with the backend interfaced with the Qwen3.5-plus large language model. We established strict environmental initialization protocols: the Memory storage is cleared prior to each experiment; a manual command is issued at the start of each dialog round to force-load the skill file, constraining the LLM within a predefined skill space; and the number of trials per dialog round adaptively limited according to task type based on task type (5–8 trials for short-cycle tasks and 1–3 trials for long-cycle tasks) to mitigate performance degradation caused by long contexts. To comprehensively evaluate the performance of the VigiClaw framework,

we constructed a simulation experimental scenario comprising 8 tasks (shown in Fig. 5).

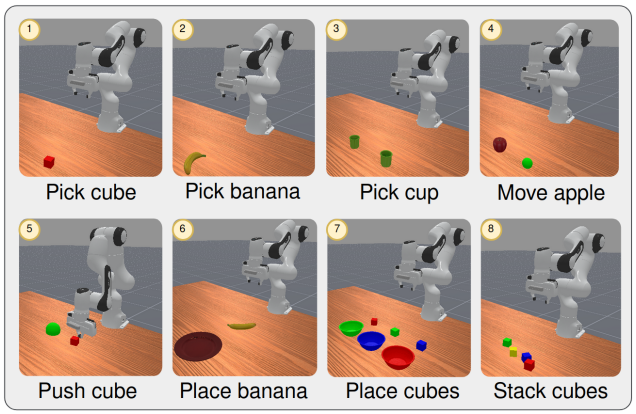


Fig. 5. The 8 tasks involved in this paper and their success criteria are as follows: Task 1 requires the red block to be lifted a certain distance (0.2 m, hereinafter the same); Task 2 requires the banana to be lifted a certain distance; Task 3 requires grasping the target cup and lifting it a certain distance; Task 4 and Task 5 require the distance error between the apple/block and the target position to be below a set threshold (0.02 m), respectively; Task 6 requires the distance between the banana and the plate center to be below a threshold; Task 7 requires blocks to be placed into bowls of matching colors; and Task 8 requires blocks to be stacked strictly in the order of green, red, blue, and yellow.

Random initialization was applied to the scene configuration and object positions in all tasks. Furthermore, task instructions were randomized based on specific task characteristics: for the cup grasping task, a direction (left or right) was randomly selected; for the apple moving and block pushing tasks, one of four directions was chosen with the distance randomized within a specified range; in the banana-to-plate task, the relative positions of the two objects were randomized; and in the block-to-bowl task, the pairing relationship between the blocks and bowls was randomly shuffled.

For long-cycle Tasks 7 and 8, we designed perturbed variants incorporating external disturbances to evaluate VigiClaw robustness. The disturbance settings are as follows: In Task 7, after a block is placed in a bowl, it is moved back to its original position, or a randomly selected empty bowl is shifted by 0.1 m; In Task 8, after placing the red block on the green block or the blue block on the red block, the upper block is perturbed and displaced back to the tabletop. Exactly one type of disturbance was applied in each execution of the task, with equal probability assigned to each type. Disturbed tasks are labeled 7* and 8*.

To evaluate the fundamental capabilities of VigiClaw, we disabled the visual detection module and tested VigiClaw on 8 standard tasks. To comprehensively evaluate the robustness to disturbance capability of VigiClaw, we conducted comparative experiments on disturbed tasks 7* and 8*. We designed three experimental groups: the full VigiClaw configuration, a configuration without visual detection, and a configuration that directly relies on ground-truth object coordinates provided by the simulator.

B. Experimental Results and Analysis

TABLE I
EXPERIMENTAL RESULTS ON STANDARD TASKS

Task ID	Task Name	Success Count	Success Rate
1	Pick Cube	25	100%
2	Pick Banana	24	96%
3	Pick Cup	23	92%
4	Move Apple	24	96%
5	Push Cube	23	92%
6	Place Banana	25	100%
7	Place Cubes	23	92%
8	Stack Cubes	19	76%

Baseline denotes the configuration without state verification. Each task is executed 25 times.

TABLE II
EXPERIMENTAL RESULTS ON DISTURBED TASKS

Group	Task Name	Success Count	Success Rate
NVD	Place cubes	0	0%
	Stack cubes	0	0%
VD	Place cubes	21	84%
	Stack cubes	18	72%
SIM	Place cubes	22	88%
	Stack cubes	21	84%

NVD: No visual detection under disturbances. VD: Full visual detection under disturbances. SIM: Object coordinates directly obtained from the simulator for state verification. Each task is executed 25 times.

As shown in Table I, the system achieves an overall success rate exceeding 92% in 8 tasks without disturbances. Despite the skill library containing only basic primitives, the system successfully performs complex tasks via compositional generalization. Manual inspection of execution videos reveals that task failures are predominantly caused by grasping inaccuracies arising from visual localization errors, which directly compromise grasp point selection. This issue is further exacerbated in the block stacking task by severe occlusions that hinder target localization, in addition to challenges posed by structural instability, which may lead to physical collapses. In contrast,

reasoning errors within the LLM's decision-making process account for only a small fraction of the observed failures.

As indicated in Table II, under the NVD (No Visual Detection) configuration, the success rates for tasks 7* and 8* were both 0%, indicating that systems lacking state feedback fail immediately after encountering disturbances. In contrast, the VD (Visual Detection) configuration achieved success rates of 84% and 72%, respectively. Although VD achieves slightly lower success rates than SIM, an oracle-state baseline that uses perfect simulator-derived coordinates, both configurations retain the full OpenClaw-centric execution loop and identical recovery procedures. Nevertheless, VD significantly outperforms NVD, confirming the effectiveness of visual detection. The VigiClaw anomaly detection and recovery process is illustrated in Fig. 6.

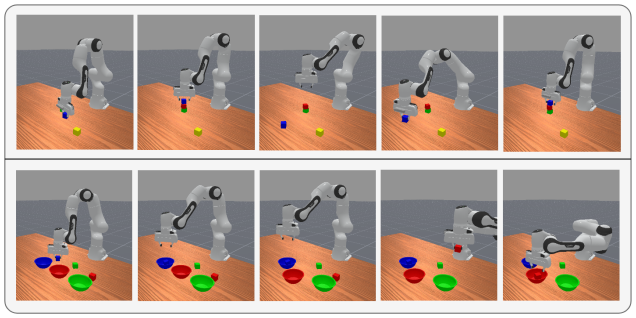


Fig. 6. The process of anomaly detection and recovery. For displaced blocks, VigiClaw initiates a re-grasp-and-place sequence; for shifted bowls, it updates the target pose using the latest detected coordinates.

IV. Conclusions

This paper presents VigiClaw, a robotic manipulation framework designed to mitigate the fragility of long-horizon task execution in dynamic environments. This framework integrates an Action-Triggered State Verification mechanism into an OpenClaw-centric architecture to achieve robust responses to external disturbances. We validated the effectiveness of this method through simulation experiments in the ManiSkill simulator. Under external disturbances, VigiClaw achieved success rates of 84% and 72% on long-horizon tasks, compared to 0% for baseline without visual detection. While the current evaluation is confined to simulation, future work will focus on real-world deployment and enhancing VigiClaw's robustness. We plan to integrate multi-view detection and object-dimension-based threshold optimization to improve perception accuracy. Furthermore, we aim to augment the system with LLM reasoning to generate flexible recovery strategies beyond pre-defined rules, thereby improving adaptability in unstructured environments.

References

[1] J. Liang, W. Huang, F. Xia, et al., "Code as policies: Language model programs for embodied control," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pp. 9493–9500, 2023.

[2] A. Brohan, Y. Chebotar, C. Finn, et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. Conf. Robot Learn. (CoRL)*, pp. 287–318, 2023.

[3] B. Zitkovich, T. Yu, S. Xu, et al., "Rt-2: Vision-language-action models transfer web knowledge to robotic control," in *Proc. Conf. Robot Learn. (CoRL)*, pp. 2165–2183, 2023.

[4] Octo Model Team, D. Ghosh, H. Walke, et al., "Octo: An open-source generalist robot policy," *arXiv:2405.12213*, 2024.

[5] K. Black, N. Brown, D. Driess, et al., "π_0: A vision-language-action flow model for general robot control," *arXiv:2410.24164*, 2024.

[6] E. Zhou, Q. Su, C. Chi, Z. Zhang, Z. Wang, T. Huang, L. Sheng, and H. Wang, "Code-as-monitor: Constraint-aware visual programming for reactive and proactive robotic failure detection," in *Proceedings of the Computer Vision and Pattern Recognition Conference*, 2025, pp. 6919–6929.

[7] P. Pacaud, R. Garcia, S. Chen, and C. Schmid, "Guardian: Detecting Robotic Planning and Execution Errors with Vision-Language Models," *arXiv:2512.01946*, 2025.

[8] E. O. P. Steinberger *et al.*, "OpenClaw: An open-source autonomous ai agent framework," 2026. [Online]. Available: https://github.com/openclaw/openclaw

[9] R. Zhao, X. Zhang, Z. Guo, et al., "ROSClaw: A hierarchical semantic-physical framework for heterogeneous multi-agent collaboration," *arXiv:2604.04664*, 2026.

[10] H. Li, X. Cao, Z. Zeng, et al., "OpenGo: An OpenClaw-based robotic dog with real-time skill switching," *arXiv:2604.01708*, 2026.

[11] S. Tao, F. Xiang, A. Shukla, et al., "Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai," *arXiv:2410.00425*, 2024.

[12] R. Sapkota, R. H. Cheppally, A. Sharda, et al., "YOLO26: Key architectural enhancements and performance benchmarking for real-time object detection," *arXiv:2509.25164*, 2025.

[13] S. Schaal, "Dynamic movement primitives-a framework for motor control in humans and humanoid robotics," in *Adaptive Motion of Animals and Machines*, pp. 261–280, Springer, 2006.

[14] A. J. Ijspeert, J. Nakanishi, H. Hoffmann, et al., "Dynamical movement primitives: Learning attractor models for motor behaviors," *Neural Comput.*, vol. 25, no. 2, pp. 328–373, 2013.

[15] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models," *arXiv:2307.05973*, 2023.

[16] H. Abdi and L. J. Williams, "Principal component analysis," *Wiley Interdisciplinary Reviews: Computational Statistics*, vol. 2, no. 4, pp. 433–459, 2010.